SQL

FUNCTIONS

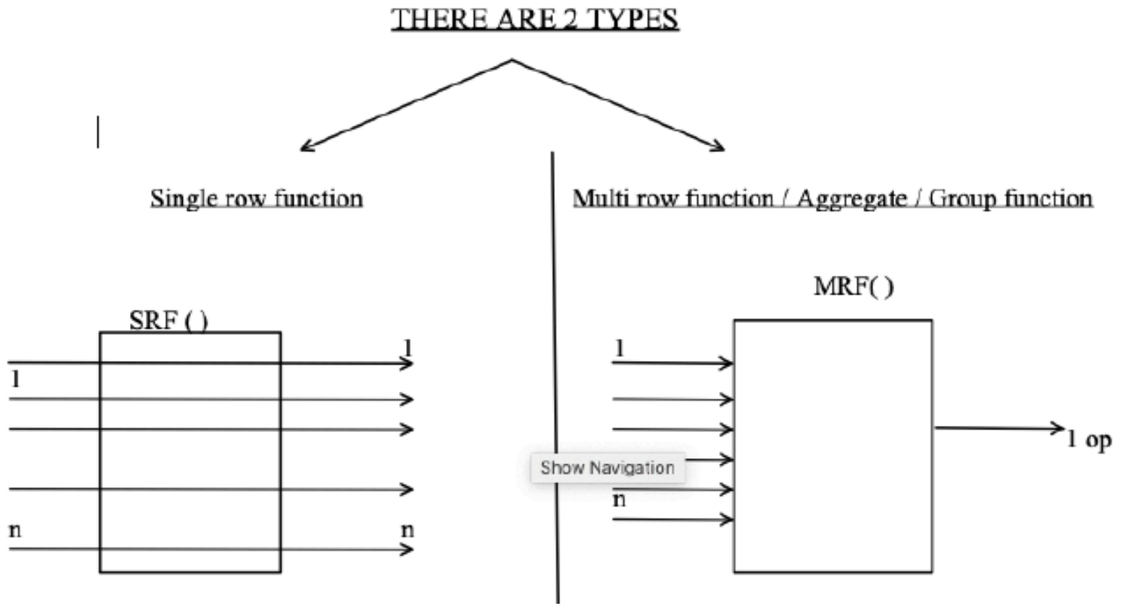
Are a block of code or list of instructions which are used to perform a specific task .

There are 3 main components of a function

1. Function_Name
2. Number_of_arguments (no of inputs)
3. Return type

Types of Functions in SQL :

1. SINGLE ROW FUNCTIONS
2. MUTLI ROW FUNCTIONS / AGGREGATE / GROUP FUNCTIONS.



2. MIN ( ) : it is used to obtain the minimum value present in the column
3. SUM ( ) : it is used to obtain the summation of values present in the column
4. AVG( ) : it is used to obtain the average of values present in the column
5. COUNT( ) : it is used to obtain the number of values present in the column

NOTE :

- Multi row functions can accept only one argument , i.e a Column_Name or an Expression

  MRF (Column_Name / Exp)

- Along with a MRF( ) we are not supposed to use any other Column_Name in the select clause .
- MRF( ) ignore the Null .
- We cannot use a MRF( ) in where clause .
- COUNT( ) is the only MRF which can accept * as an Argument .

GROUP & FILTERING

GROUPING : GROUP BY Clause

Group by clause is used to *group the records* .

SYNTAX:

```
SELECT  group_by_expression / group_function
FROM table_name
[WHERE <filter_condition>]
GROUP BY column_name/expression ;
```

ORDER OF EXECUTION:

1-FROM
2-WHERE(if used) [ROW-BY-ROW]
3-GROUP BY [ROW-BY-ROW]
4-SELECT [GROUP-BY-GROUP]

NOTE :

- Group By clause is used to group the records .
- Group By clause executes row by row .
- After the execution of Group By clause we get Groups .
- Therefore any clause that executes after group by must execute Group By Group .
- The Column_Name or expression used for grouping can be used In select clause .
- Group By clause can be used without using Where clause .

FILTERING : HAVING Clause

" Having Clause is used to Filter the Group "

SYNTAX:

```
SELECT  group_by_expression / group_function
FROM table_name
[WHERE <filter_condition>]
GROUP BY column_name/expression
HAVING  <group_filter_condition>
```

ORDER OF EXECUTION:

1-FROM	
2-WHERE(if used)	*[ROW-BY-ROW]*
3-GROUP BY(if used)	*[ROW-BY-ROW]*
4-HAVING (if used)	*[GROUP-BY-GROUP]*
5-SELECT	*[GROUP-BY-GROUP]*

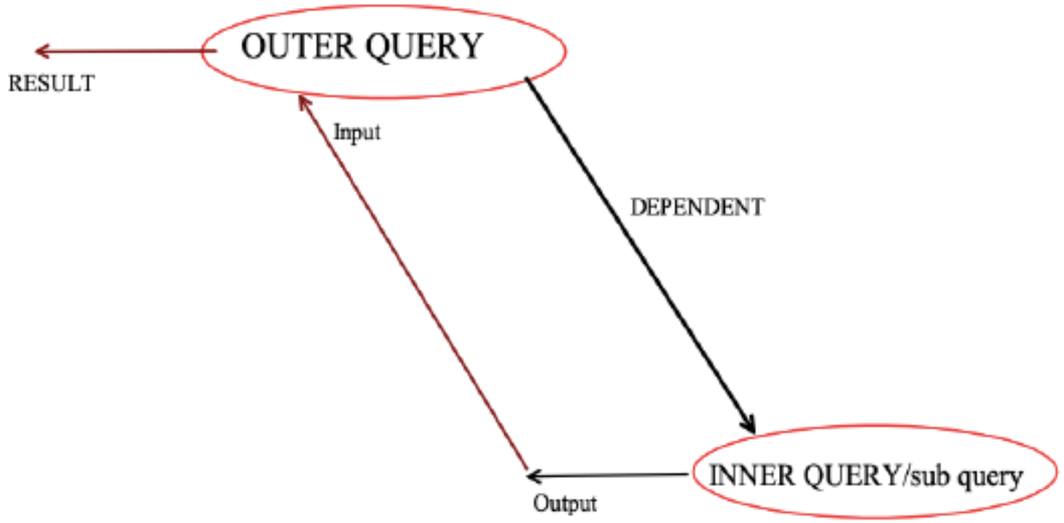
Differentiate between Where and Having .

WHERE	**HAVING**
➢ Where clause is used to Filter the records	➢ Having clause is used to Filter the groups .
➢ Where clause executes row By row .	➢ Having clause executes Group by group
➢ In Where Clause we cannot Use MRF( )	➢ Can use MRF( ).
➢ Where clause executes before Group by clause .	➢ Having clause executes After group by clause .

SUB-QUERY

" A QUERY WRITTEN INSIDE ANOTHER QUERY IS KNOWN AS SUB QUERY "

Working Principle :



Let us consider two queries Outer Query and Inner Query .

- Inner Query executes first and produces an Output .
- The Output of Inner Query is given / fed as an Input to Outer Query .
- The Outer Query generates the Result.
- Therefore we can state that 'the Outer Query is dependent on Inner Query' and this is the Execution Principle of Sub Query .

Case 1 : Whenever we have Unknowns present in the Question We use sub query to find the Unknown .

Questions

WAQTD names of the employees earning more than 2500 .

WAQTD names of the employees earning less than MILLER .

WAQTD name and deptno of the employees working in the same Dept as SMITH .

WAQTD name and hiredate of the employees if the employee Was hired after JONES .

WAQTD all the details of the employee working in the sameDesignation as SCOTT .

WAQTD name , sal , deptno of the employees if the employees Earn more than 2000 and work in the same dept as JAMES .

WAQTD all the details of the employees working in the Same designation as MILLER and earning more than 1500.

WAQTD details of the employees earning more than SMITH But less than KING .

WAQTD name and hiredate of the employees who's name ends with 'S' and hired after James .

CASE-2 : Whenever the data to be selected and the condition to be executed are present in different tables we use Sub Query .

Questions

WAQTD deptname and location of the employee whose name is Miller .

WAQTD dname of the employee whose name is Miller .

WAQTD Location of ADAMS

WAQTD names of the employees working in Location CHICAGO.

TYPES OF SUB QUERY :

1. **Single Row Sub Query :**

- **If a sub query returns exactly 1 record / 1 value We call it as Single Row Sub Query .**
- If a sub query returns single value then we can use both Normal Op as well as Special Op .

2. **Multi Row Sub Query :**

- **If a sub query returns more than 1 record / 1 value We call it as Multi Row Sub Query .**
- If a sub query returns more than 1 value then we cannot use Normal Op we *must only use Special Op* .

Subquery Operator

1. ALL Operator :

- *All operator is a special operator which has to be used along with relational Operator which will compare the value present at the LHS with all the values of RHS*
- *>ALL, <ALL , >=ALL, <=ALL*
- *ALL op* ***returns true*** *if all the values at the RHS have satisfied the Condition .*

2. ANY Operator :

- *Any operator is a special operator which has to be used along with relational Operator which will compare the value present at the LHS with all the values of RHS*
- *ANY op* ***returns true*** *if one of the values at the RHS have satisfied the Condition .*

NESTED SUB QUERY

- A sub query written inside another sub query is known as Nested sub query .
- We can nest about 255 sub queries .

Questions on nested subquery

- WAQTD the maximum salary given to the employees .
- WAQTD second maximum salary .
- WAQTD Third maximum salary .
- WAQTD 3 rd minimum salary .
- WAQTD name of the employee getting 2nd Max salary .
- WAQTD Dname of the employee getting 2^{ND} MAX salary .
- WAQTD Dname of the employee getting 3RD minimum salary .

EMPLOYEE - MANAGER RELATION .

CASE 1 : TO FIND THE MANAGER OF AN EMPLOYEE

- WAQTD the manager name of Allen .
- WAQTD the name of JONES' s manager .
- WAQTD Dname of Blake's Manager .
- WAQTD Allen's Manager's manager name .

- **CASE 2 : TO FIND THE EMPLOYEE REPORTING TO MANAGER**
- WAQTD names of the employees reporting to Blake .
- WAQTD salary of the employee reporting to KING .

JOINS

Types of JOINS .

We have 5 types of joins

- CARTESIAN JOIN / CROSS JOIN
- INNER JOIN / EQUI JOIN
- OUTER JOIN
    - i. LEFT OUTER JOIN
    - ii. RIGHT OUTER JOIN
    - iii. FULL OUTER JOIN
- SELF JOIN
- NATURAL JOIN

CARTESIAN JOIN / CROSS JOIN :

- **In Cartesian Join a record from table 1 will be merged with All the records of table 2 .**

SYNTAX:

1. ANSI [American National Standard Institute]

```
SELECT Column_Name
FROM Table_Name1 CROSS JOIN Table_Name2 ;
```

2. Oracle

```
SELECT Column_Name
FROM Table_Name1 , Table_Name2 ;
```

2. INNER JOIN :

- **" It is used to Obtain only Matching Records ".**
- **SYNTAX:**

1. ANSI [American National Standard Institute]

SELECT Column_Name

FROM Table_Name1 **INNER JOIN** Table_Name2

ON < JOIN_CONDITION> ;

- **2. Oracle**

SELECT Column_Name

FROM Table_Name1 , Table_Name2

WHERE <JOIN_CONDITION > ;

- **JOIN Condition :** It is a condition on which the two tables Are merged .

  Syntax: Table_Name1.Col_Name = Table_Name2.Col_Name

  eg. **Join Condition :EMP.DEPTNO = DEPT.DEPTNO**

- WAQTD ename and dept name for all the employees .
- WAQTD ename and loc for all the employees working as Manager .
- WAQTD ename , sal and dname of the employee working as Clerk in dept 20 with a salary of more than 1800
- WAQTD ename deptno , dname and loc of the employee earning more than 2000 in New York .

3. NATURAL JOIN :

- "It behaves as INNER JOIN if there is a relation between the
- given two tables , else it behaves as CROSS JOIN" .

Syntax:

- **ANSI :**
- SELECT Col_Name
- FROM Table_Name1 NATURAL JOIN Table_Name2;

OUTER JOIN

- **" It is used to Obtain Un-Matched Records Along with Matching Records"**

1. Left Outer Join :

- **" It is used to obtain Un-Matched Records of Left Table Along with Matching Records" .**

Syntax:

1. ANSI [American National Standard Institute]

```
SELECT Column_Name
FROM Table_Name1 LEFT [OUTER] JOIN Table_Name2
ON < JOIN_CONDITION> ;
```

2. Oracle

```
SELECT Column_Name
FROM Table_Name1 , Table_Name2
WHERE Table1.Col_Name = Table2.Col_Name (+) ;
```

RIGHT Outer Join :

- **" It is used to obtain Un-Matched Records of Right Table Along with Matching Records" .**

Syntax:

1. ANSI [American National Standard Institute]

```
SELECT Column_Name
FROM Table_Name1 RIGHT[OUTER] JOIN Table_Name2
ON < JOIN_CONDITION> ;
```

2. Oracle

```
SELECT Column_Name
FROM Table_Name1 , Table_Name2
WHERE Table1.Col_Name (+) = Table2.Col_Name ;
```

- **3. Full Outer Join :**
- **" It is used to obtain Un-Matched Records of both Left**
- **& Right Table Along with Matching Records" .**
- **ANSI** [American National Standard Institute]

SELECT Column_Name

FROM Table_Name1 **FULL [OUTER] JOIN** Table_Name2

ON < JOIN_CONDITION> ;

- **SELF JOIN :**
- **"Joining a table by itself is known as Self Join "**

Why ? / When ? - "Whenever the data to select is in the same table but present In different records we use self join ".

- **SYNTAX:**

1. ANSI [American National Standard Institute]

SELECT Column_Name

FROM Table_Name1 **JOIN** Table_Name2

ON < JOIN_CONDITION> ;

2. Oracle

SELECT Column_Name

FROM Table_Name1 T1 **,** Table_Name2 T2

WHERE < Join_Condition > ;